

Purpose of This Document

This is a record of every real error encountered while setting up and running this project, in the order they happened. Keeping this kind of log is genuine professional practice — it documents troubleshooting ability, which is often more valuable to demonstrate than a project that “just worked.”

Issue 1 — `pip install` failed to build `pydantic-core`

Error:

```
ERROR: Failed building wheel for pydantic-core
Failed to build pydantic-core
```

Cause: Python 3.14 was very newly released. Several packages, including `pydantic-core`, didn't yet have a pre-built installer file (“wheel”) for this exact Python version, so `pip` tried to compile it from source code — which failed without extra build tools installed.

Fix: Removed exact version numbers (`==x.x.x`) from `requirements.txt`, letting `pip` install whichever version *did* already have a pre-built file for Python 3.14, instead of forcing an old exact version that only shipped source code.

Lesson: Pinning exact versions is good practice generally, but very new Python releases can briefly lag behind on ecosystem support. Unpinning (or pinning to a *range*) is a reasonable temporary workaround.

Issue 2 — MySQL connection failed: `getaddrinfo` failed

Error:

```
socket.gaierror: [Errno 11003] getaddrinfo failed
pymysql.err.OperationalError: (2003, "Can't connect to MySQL server on
'Nand23102002@localhost' ...")
```

Cause: The database connection string follows the format `username:password@host`. The MySQL password itself contained an `@` character, which broke the parsing — everything after the *first* `@` was misread as the hostname, so it tried to connect to a server literally named `Nand23102002@localhost`.

Fix (chosen approach): Changed the actual MySQL password to remove the `@` character entirely, via:

```
ALTER USER 'root'@'localhost' IDENTIFIED BY 'Nand23102002';
```

(An alternative fix, without changing the password, would have been to URL-encode the `@` inside the connection string as `%40`.)

Lesson: Special characters in credentials can conflict with the syntax of the format they're embedded in. Either avoid them in service passwords, or know how to encode them.

Issue 3 — `Unknown database 'expense_db'`

Error:

```
pymysql.err.OperationalError: (1049, "Unknown database 'expense_db'")
```

Cause: The `CREATE DATABASE expense_db;` step had been interrupted earlier (while debugging Issue 2) and never actually completed, so the database the project expected to connect to didn't exist yet.

Fix: Logged into MySQL directly and ran:

```
CREATE DATABASE expense_db;
SHOW DATABASES; -- confirmed it now appears in the list
```

Lesson: After fixing one error, it's worth confirming each *earlier* step actually completed too — fixes can get interrupted mid-way without it being obvious.

Issue 4 — Login form 422 Validation Error : wrong grant_type

Error:

```
{
  "msg": "String should match pattern '^password$',
  "loc": ["body", "grant_type"]
}
```

Cause: The login form's `grant_type` field is a fixed keyword the system requires (literally the word `password`) — it was mistakenly filled in with the actual account password instead.

Fix: Set `grant_type` to the literal text `password`, and put the real credentials in the `username` / `password` fields instead.

Lesson: Some form fields are protocol-level keywords, not user-data — reading field names/patterns carefully matters, especially with unfamiliar login standards (this one follows the OAuth2 “password flow” convention).

Issue 5 — 500 Internal Server Error on registration: bcrypt version conflict

Error:

```
AttributeError: module 'bcrypt' has no attribute '__about__'
...
ValueError: password cannot be longer than 72 bytes, truncate manually if
necessary
```

Cause: Because package versions were unpinned (to solve Issue 1), a newer version of the `bcrypt` library was installed than the `passlib` library (used for password hashing) knew how to talk to — a known compatibility break between these two specific packages' recent versions.

Fix:

```
pip install "bcrypt==4.0.1"
```

Installing this specific older version of just `bcrypt` resolved the conflict without needing to touch anything else.

Lesson: Unpinning *all* versions to fix one compatibility issue (Issue 1) can introduce a *different* compatibility issue elsewhere. Sometimes the right fix is pinning just the one problem package, not leaving everything unpinned.

Issue 6 — Confusion over client_id / client_secret fields in login

Cause: Not an error exactly, but a point of confusion: the login form displayed extra fields (`client_id`, `client_secret`, `scope`) that appeared required-looking but weren't used by this project at all.

Resolution: Confirmed these are part of a more advanced OAuth2 setup (used for third-party application logins) that this project's simple username/password flow doesn't use — left blank with no ill effect.

Lesson: Auto-generated forms sometimes show more fields than a specific implementation actually needs. It's fine (and correct) to leave protocol fields blank when they don't apply.

Summary Table

#	Issue	Category	Fixed by
1	pip wheel build failure	Environment/versioning	Unpinned requirements.txt
2	MySQL connection string broken by @ in password	Configuration	Changed password / would also work with URL-encoding
3	Database didn't exist	Setup sequencing	Re-ran CREATE DATABASE
4	Wrong value in grant_type	User input/misunderstanding	Corrected form field
5	bcrypt/passlib version conflict	Dependency compatibility	Pinned bcrypt==4.0.1
6	Unused OAuth2 fields caused confusion	Understanding	Left blank, confirmed not needed

Every one of these is a legitimate, common real-world debugging scenario — none of them reflect a flaw in understanding, only the normal friction of setting up a new backend environment for the first time.

End of Issues Log.